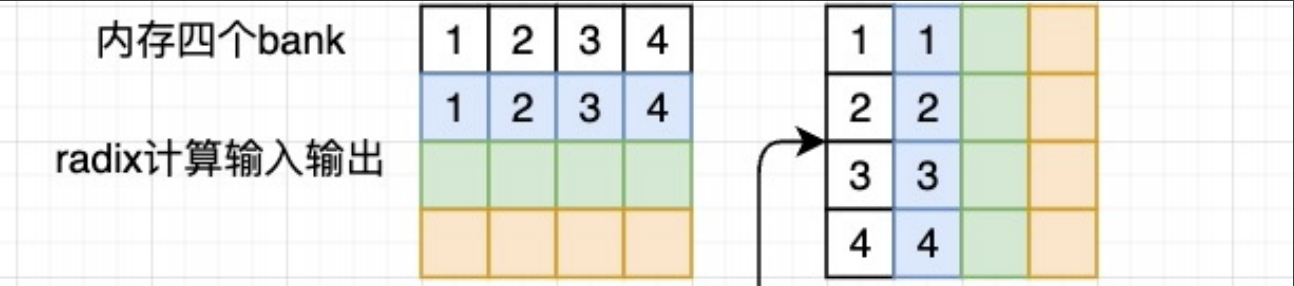


无冲突存储访问算法

首先，我们这样表示一个radix-4 16点的算法，并且只有1个radix-4单元，通过复用计算和从DRAM/SRAM交互中间数据，完成16点运算。假设我们的DRAM/SRAM存在4个独立的channel/bank，意味着每个channel/bank可以独立读写，那我们希望实现一种无冲突的存储访问算法，它的要求是：

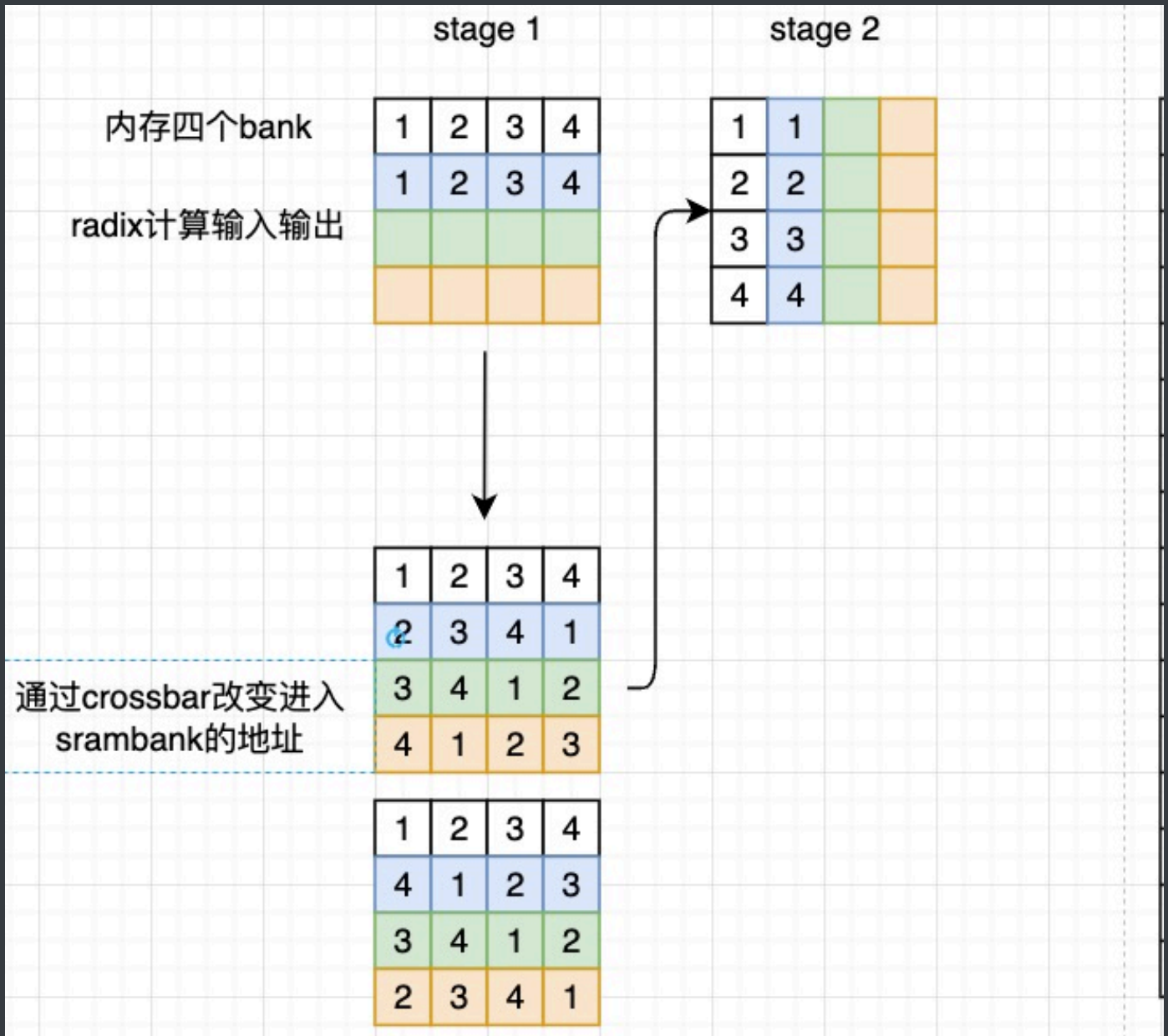
- 1. 每次读的时候，radix-4单元需要的四个数据能够正好分别从四个通道读入
- 每次写的时候，radix-4单元产生的四个数据能够正好分别从四个通道写入
- 这样就可以实现的无冲突的意思就是不会因为读写的四个数据只在在1~3个存储通道中而造成堵塞。

比方说，下面的二维方格表示一个需要处理的16点数据，在第一个阶段里，1个radix-4单元需要做四次行变换，分别用白蓝绿黄表示顺序。格子中的数字表示数据存在的bank1~4。那么，在读取的时候，假设数据如此放置，那么每次读都是从四个通道读取数据，内存无冲突。



在第一个阶段计算完写入数据，在第二个阶段读数据的时候，如果数据排版还是采用和刚刚一样，那么radix-4做列变换时候，就只能从1个通道中读取四个数据，那么需要更长的周期，冲突发生。

为了解决这个问题，我们在第一个阶段计算完写入数据时，通过互联操作，使得数据重新排列如下：



很明显，在这种情况下，第二个阶段的读取无冲突发生。而且重新排列的方案不是唯一的，上图就列了两种。

刚刚的表示方法重视展示每次计算涉及的数据，和bank划分之间的关系。现在我们换一种可视化方法，重点表示bank划分和bank地址。

这回我们假设时64点radix-4，一个radix-4单元，bank=4。表格纵坐标表示地址，横坐标表示bank序号，数字表示数据的序号。按照刚刚讨论的策略，第一阶段前和第一阶段后第二阶段前数据排布如下，图1第一阶段前 图2第二阶段后：



我们同样用颜色表示从存储中读取的顺序，涂色，就是图3 4。

我们发现，所有的通道都是按照灰红绿蓝的顺序进行读取的，也就是全都是顺序读取+连续地址！这也就是说可以实现burst在DRAM上。

现在我们看TCAS1 那张图：这张图是radix-16 2^{16} 点的memory映射。16bank，一个radix-16单元。是不是和我们的方法一样？

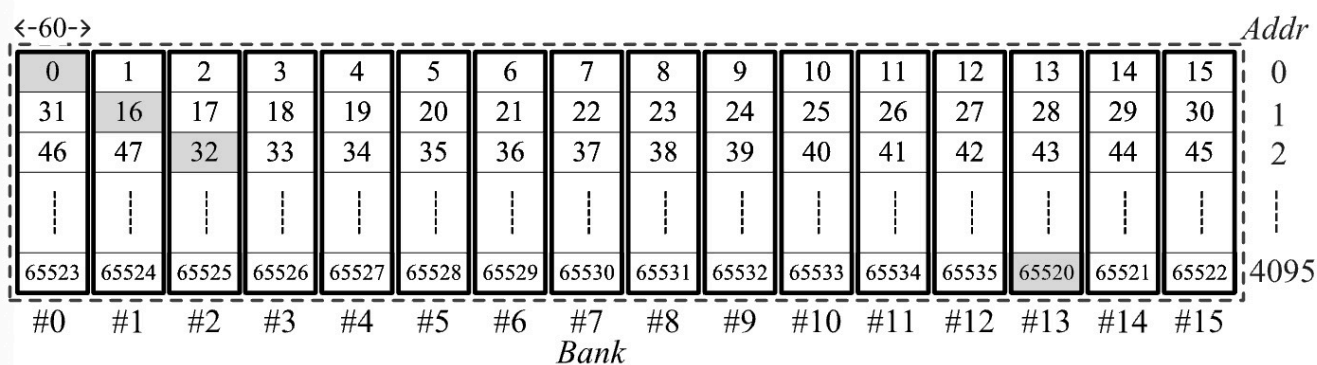


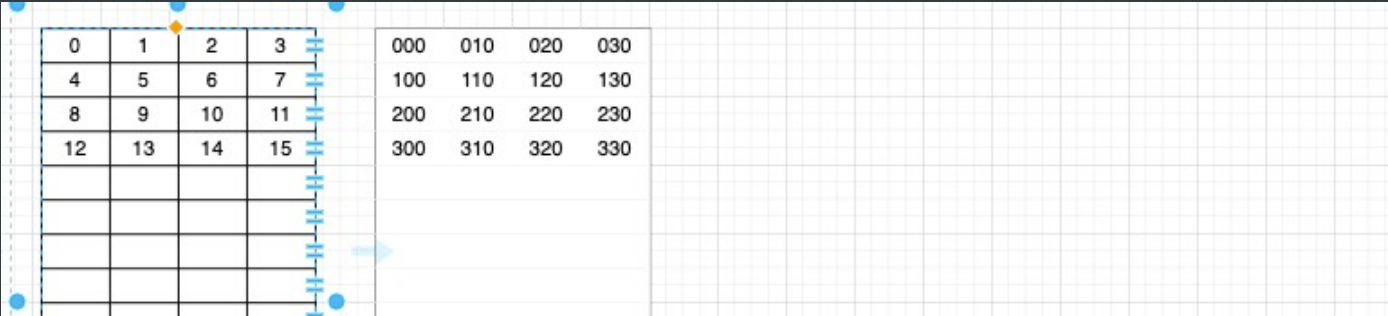
Fig. 6. DM address scheme is same for NTT and INTT architectures. The DM unit constructs 16 memory banks in parallel. Indices indicate the original orders of coefficients in polynomial. The location of coefficient is determined by the conflict-free address mapping scheme.

下面升级两个问题：

1. 刚刚讨论的都是第一个阶段和第二个阶段，，后面的阶段不是跨度更大吗要如何使用呢

很简单，我们假设第二阶段算完后有一个原生的memory排布，指的是假设直接把radix-4输出导存储中，那么我们把它写成4进制的情况下是这个样子的。

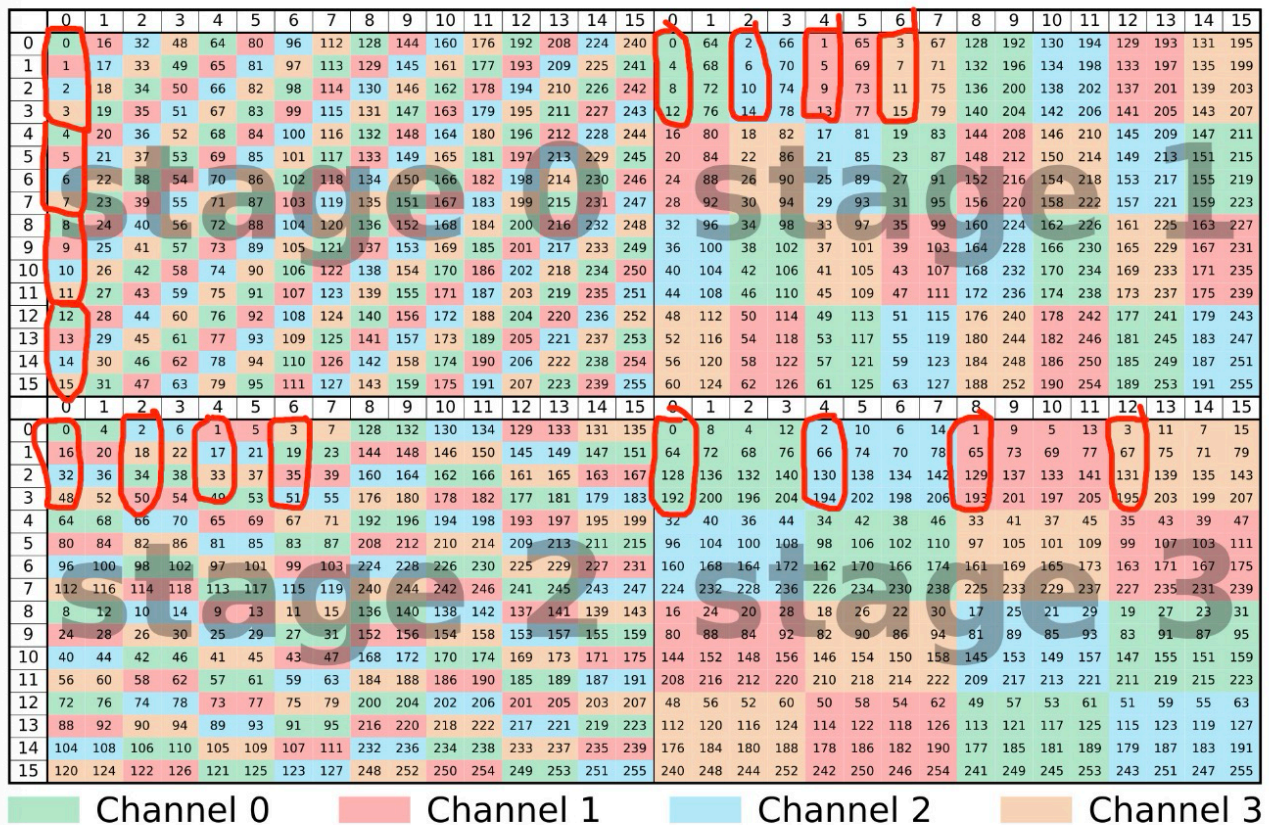
左边：第一阶段输入 右边：第二阶段输出



明显，二阶段->三阶段的映射问题和一阶段->二阶段的映射问题极为相似。你定量解决一下。

2. 刚刚讨论的情况是bank分块和radix单元的点数一样。如果类似DAC zhangjiliang那篇图4的情况，bank=4，radix-4，但是4个radix-4单元怎么办（一次要读16个数可是只有4个bank）

这回无论哪个阶段，都至少需要4次从4个bank的存储中读取，才能获得16个数。也就是说，只要每次读取的时候每个bank都是读出四个数字就行，而且连续读取的时候地址连续。我们看他的图，是不是符合这个约束？



分布均匀无conflict好看出来，每次读的16个数都刚好每个channel 4个。地址连续可以burst这个结合上下阶段也可以看出来。能发现在stage0-1，stage2-3这两个阶段的地址控制还变简单了。

结论：我们发现只要满足无冲突访存约束就能归纳出不唯一的方案。tcas1和dac这两篇的访存方案都有mod和线性加法项，这样能做出一个循环移位的效果，容易实现无冲突访存和burst访存。可以先画画图然后归纳一下，再看看归纳的方法符合约束不，符合就ok了。他们最后规划出的公式很难看，不如自己归纳一个知道是咋来的。